DESIGN AND ANALYSIS OF ALGORITHMS · UNIT 2

Quick Sort

Divide and conquer, where the partition does the work

Ammey Kuraaday
Roll No. B107

Shreyash Javale
Roll No. B34

OVERVIEW

What we will cover

01

The idea

Divide and conquer without a merge step

02

Partition

Pivot, two pointers and a full trace

03

Algorithm

Code, recursion and one bug

04

Analysis

Best, worst and average case

05

Real world

How libraries make it fast

WHO PRESENTS WHAT

Ammey Kuraaday
Slides 1 to 10: idea, partition, code, recursion

Shreyash Javale
Slides 11 to 19: the bug, analysis, real world

THE QUESTION

Why is quick sort everywhere?

The fast built-in sorts of C++, Go and Rust are all quick sort variants.

Merge sort guarantees O(n log n). Quick sort can fall to $O(n^2)$.

So why does quick sort win?

1959

Tony Hoare invents quick sort as a visiting student at Moscow State University, while working on a machine translation project.

Same recipe, different place for the work

	Merge sort	Quick sort
Divide	Trivial: cut the array in the middle	**The real work: partition around a pivot**
Conquer	Sort both halves recursively	Sort both parts recursively
Combine	**The real work: merge two sorted halves**	Nothing. The array is already in order

Quick sort moves the effort from combine to divide.

The two parts are independent, so nothing has to be put back together.

After one partition, the pivot is home



sorted on its own

final position

sorted on its own

1

Pick a pivot v

Here we take the first element, A[low].

2

Partition

Smaller or equal values go left, larger or equal go right.

3

Recurse on both sides

The pivot is in its final spot and never moves again.

Partition: two pointers, one swap at the end

```
 1  Partition(A, low, high)
 2    pivot = A[low]
 3    i = low + 1
 4    j = high
 5    while true
 6      while A[i] <= pivot: i++
 7      while A[j] >  pivot: j--
 8      if i < j: swap(A[i], A[j])
 9      else: break
10    swap(A[low], A[j])
11    return j
```

i Scans right. Stops at the first element greater than the pivot.

j Scans left. Stops at the first element not greater than the pivot.

swap If i < j, swap A[i] and A[j], then keep scanning.

end Once they cross, swap the pivot into A[j]. That index is final.

Condensed from the course notes. The exact boundary checks depend on the implementation.

Partition on [65, 70, 75, 80, 85, 60, 55, 50, 45]



The same trace, as a table

Step	i stops at	j stops at	Action	Array afterwards
start	-	-	pivot = A[0] = 65	`65 70 75 80 85 60 55 50 45`
1	`A[1] = 70`	`A[8] = 45`	i < j: swap	`65 45 75 80 85 60 55 50 70`
2	`A[2] = 75`	`A[7] = 50`	i < j: swap	`65 45 50 80 85 60 55 75 70`
3	`A[3] = 80`	`A[6] = 55`	i < j: swap	`65 45 50 55 85 60 80 75 70`
4	`A[4] = 85`	`A[5] = 60`	i < j: swap	`65 45 50 55 60 85 80 75 70`
5	`A[5] = 85`	`A[4] = 60`	i > j: swap A[0], A[j]	**`60 45 50 55 65 85 80 75 70`**

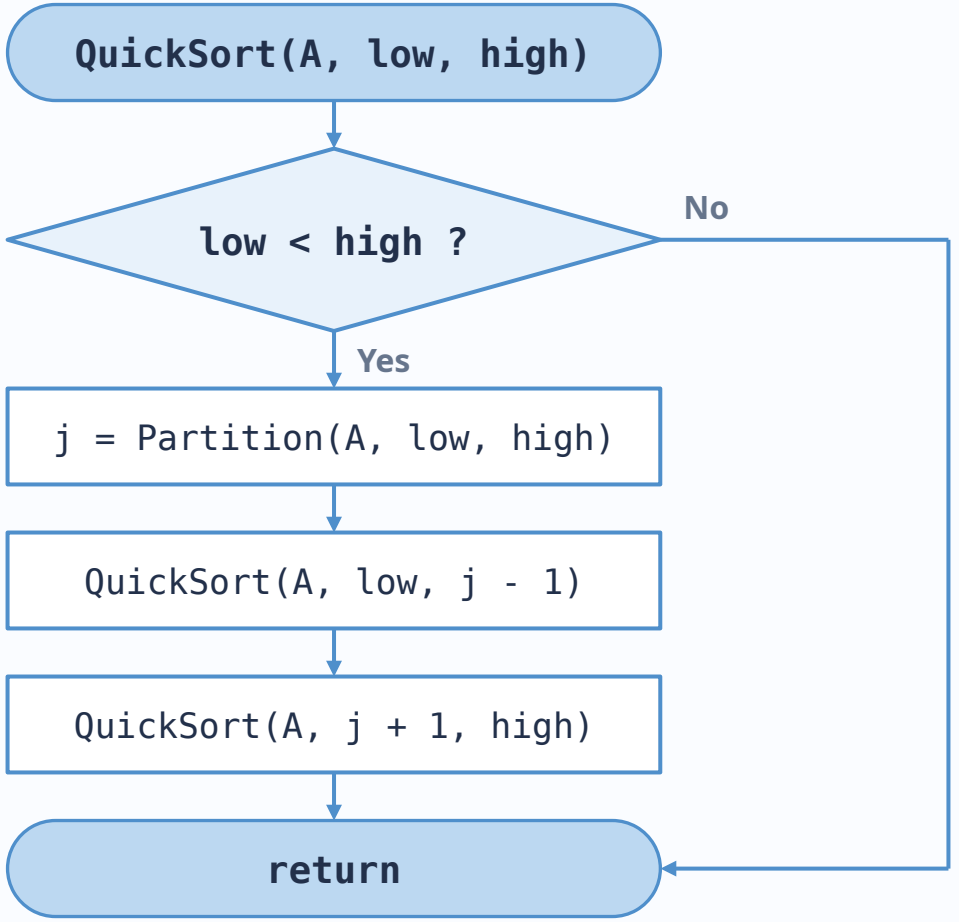
Returns j = 4. The pivot 65 is now in its final position.

QuickSort: partition, then recurse twice

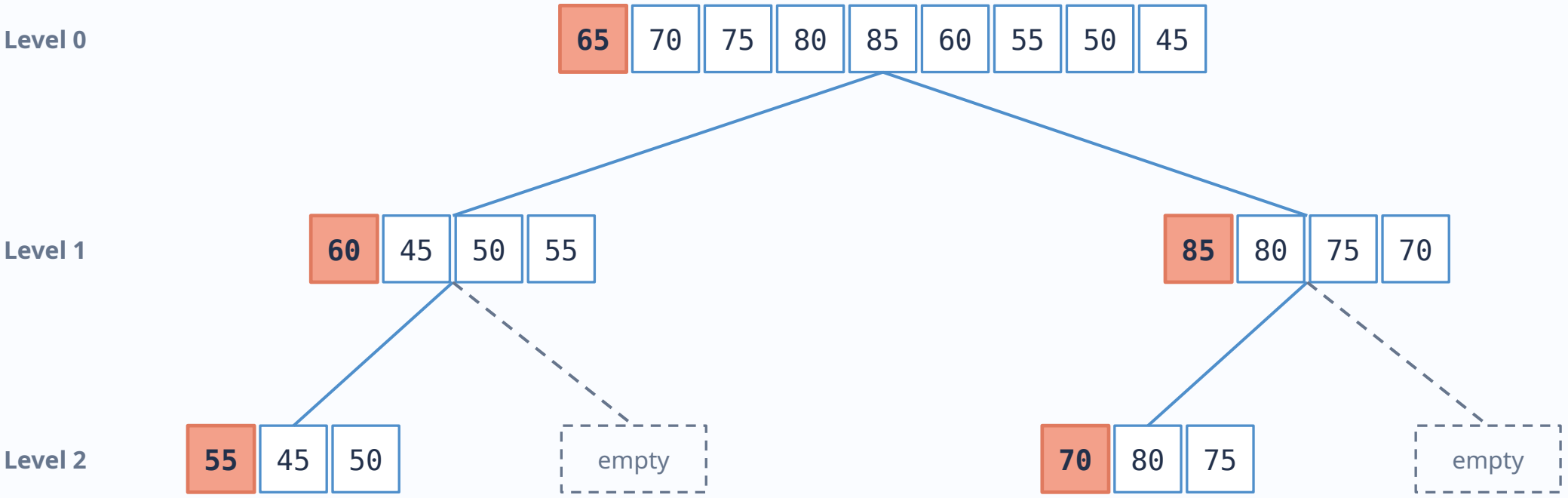
```
1  QuickSort(A, low, high)
2    if low < high
3      j = Partition(A, low, high)
4      QuickSort(A, low, j - 1)
5      QuickSort(A, j + 1, high)
```

No combine step

When both recursive calls return, the whole range is already sorted. This is the key difference from merge sort.



How the recursion unfolds



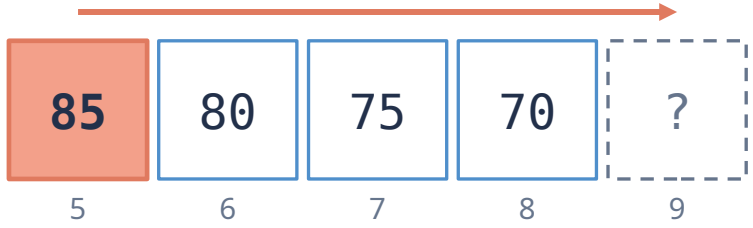
Each call fixes one pivot for good. The height of this tree decides the running time.

A boundary bug hiding in the notes

AS WRITTEN IN THE NOTES

```
while A[i] <= pivot: i++
```

i keeps moving right



Every element is ≤ 85, so i never stops. It walks off the end and reads A[9], which is not part of the array.

WITH A BOUNDS CHECK

```
while i <= high and A[i] <= pivot: i++
```

after the final swap



i now stops at high + 1. j finds the pivot, and the last swap puts 85 at index 8, its final place.

Best case: the pivot splits evenly

RECURRENCE

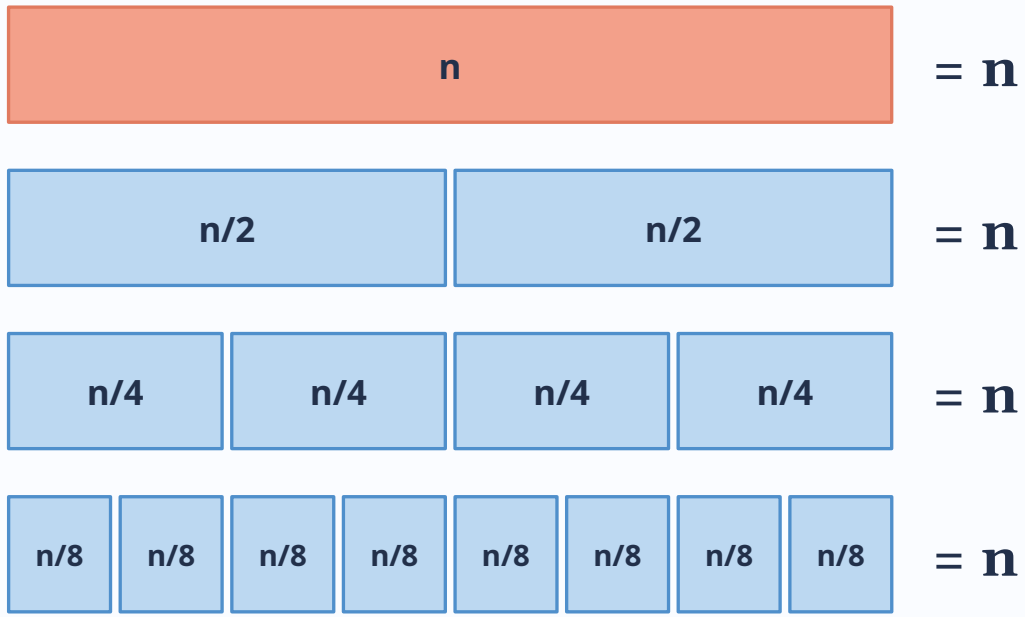
$$T(n) = 2T(n/2) + n$$

The pivot lands in the middle every time, so both parts have size n/2.

$$\Theta(n \log n)$$

log n levels (base 2), each doing n work.

WORK PER LEVEL



… down to size 1

n work × log n levels

Worst case: every split is lopsided

RECURRENCE

$$T(n) = T(n - 1) + n$$

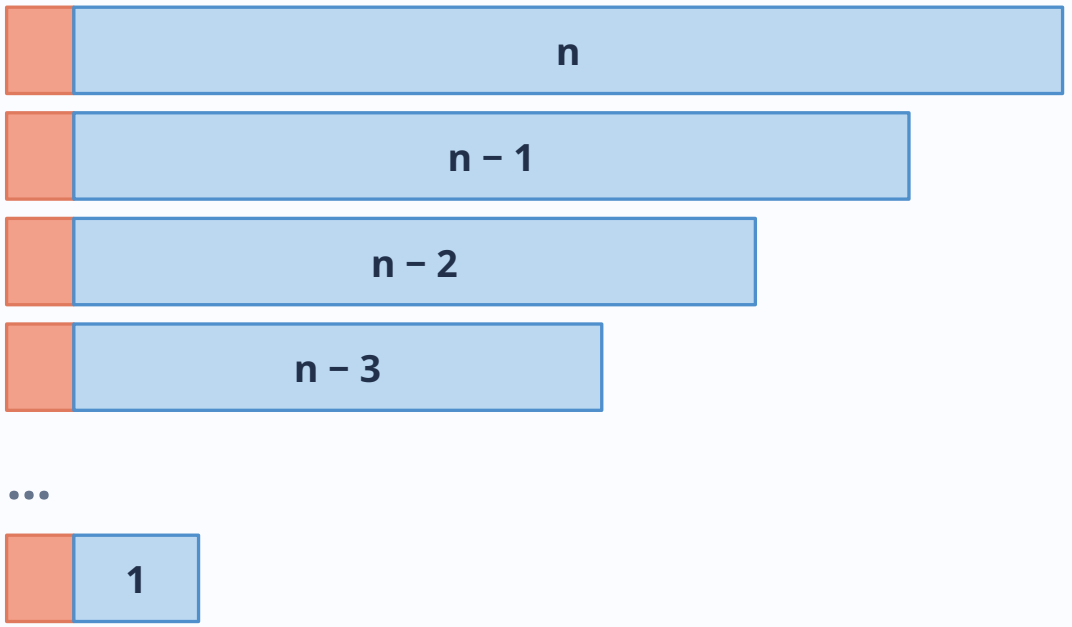
$$n + (n-1) + \dots + 1 = n(n-1)/2$$

$$\Theta(n^2)$$

n levels deep, about n/2 work on average per level

Already sorted input with a first-element pivot is the worst case.

EACH CALL REMOVES ONE ELEMENT



5,000 sorted values: 12,497,500 comparisons, against about 61,000 for n log n.

Average case: close to the best case

EXPECTED COMPARISONS

≈ 1.39 n log n

which is about 2 n ln n

Random pivot, or random input order. This is not the case where the pivot is the median: that is the best case.

Only 39% above the floor

No comparison sort can beat n log n. Quick sort is about 39% above it.

Uneven splits are fine

Even a 9 to 1 split at every level still gives O(n log n).

No bad inputs

With a random pivot the expected time is O(n log n) on every input.

Choosing the pivot

Strategy	How it works	Comparisons	Weakness
First element	Take A[low], as in our notes	≈ 1.39 n log n on random data	**Sorted input gives n^2**
Random element	Pick an index at random	≈ 1.39 n log n, any input	Needs random numbers
Median of three	Median of first, middle, last	≈ 1.19 n log n	Tricky inputs still exist
Ninther	Median of three medians of three	Splits closer to even	Extra work per partition

Quick sort against merge sort and heap sort

Algorithm	Best	Average	Worst	Extra space	Stable	In place
Quick sort	n log n	n log n	**n^2**	log n *	No	Yes
Merge sort	n log n	n log n	n log n	n	Yes	No
Heap sort	n log n	n log n	n log n	1	No	Yes

* The recursion stack, when we always recurse into the smaller part first.

Quick sort pays with a bad worst case and no stability. It gains tiny memory use and speed.

How real libraries use quick sort

Introsort

`C++ std::sort`

Starts as quick sort, switches to heap sort if recursion gets too deep, and uses insertion sort on tiny ranges.

pdqsort

`Go sort, Rust sort_unstable`

Spots bad pivots and ordered patterns, then changes strategy, so the worst case is rare.

Dual-pivot

`Java Arrays.sort`

Two pivots give three parts. Java has used it for sorting primitive arrays since version 7.

So why quick sort?

It sorts in place, reads memory in order which suits the CPU cache, has a small constant factor, and modern libraries engineer the worst case away.

SUMMARY

Key takeaways

1. Partition does the work. The pivot lands in its final position.
2. There is no combine step, unlike merge sort.
3. Best and average case are n log n. Worst case is n^2, for example sorted input with a first-element pivot.
4. Random or median-of-three pivots, and hybrids like introsort, make it the practical default.

Thank you

Questions?

Ammey Kuraaday · B107

Shreyash Javale · B34

BACKUP

Two classic partition schemes

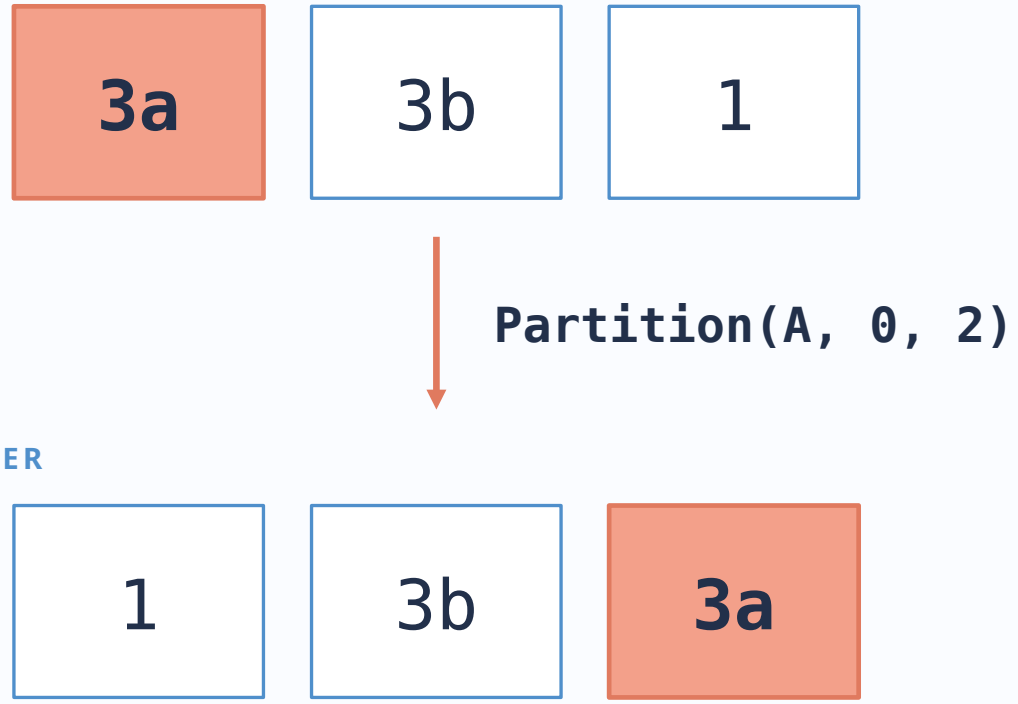
	Lomuto	Hoare (two pointers)
Scan	One pointer, left to right	Two pointers, from both ends
Typical pivot	Last element	First or middle element
Swaps	More	About 3 times fewer on average
All keys equal	**Splits n−1 : 0, so n^2**	Pointers meet in the middle, so n log n

Our notes use a Hoare-style scan with the pivot swapped into place at the end. With the $\leq$ test it also slows to n^2 when all keys are equal.

BACKUP

Why quick sort is not stable

BEFORE



What went wrong

The two equal keys 3a and 3b swapped order.

The pivot is swapped over a long distance, jumping past equal keys.

If stability matters, use merge sort or a stable library sort.